Campus Point
ENTERPRISE EDTECH ARCHITECTURE GROUP

CONFIDENTIAL TECHNICAL BLUEPRINT • VER 1.0

Automated JEE Examination, Evaluation & Analytics Platform

Comprehensive System Architecture, Relational Schema, Multi-Pattern Evaluation Engine, NTA Interface Specification, and Microsoft Power BI Enterprise Star Schema.

ENGINEERING LEADERSHIP
Senior Software Architect, DB Engineer, EdTech PM & Data Analyst

TARGET PLATFORM
Campus Point (JEE Main & JEE Advanced Ecosystem)

ARCHITECTURE STATUS
Production-Ready Masterplan • Oct 2026

Table of Contents & Architecture Index

01. Executive Summary
02. Recommended Technology Stack
03. System Architecture (Mermaid)
04. End-to-End Workflow Flowchart
05. MongoDB vs SQL Responsibility
06. SQL Schema & ER Diagram
07. Automated Paper Generation
08. NTA-Inspired Exam Interface
09. Anti-Cheating & Browser Bounds
10. Evaluation Formulas & Examples
11. Student Report Design (Deep-Blue)
12. Multi-Persona Dashboards
13. REST API Specification
14. Power BI Star Schema & DAX
15. Security, Reliability & Scale
16. Phased Implementation Roadmap
17. Testing Strategy & Checklist
18. Recommended Project Structure
19. Migration & Integration Steps
20. Prioritized Implementation Tasks

Design Paradigm: Zero regression on existing MongoDB users, enrolments, and CP-IDs. Transitioning from unstructured PDF test parsing to an ACID-compliant, bilingual, automated JEE testing and business analytics platform.

01 Executive Summary

Campus Point currently powers college cutoffs, counselling predictors, and mentorship workflows using a Node.js/Express and MongoDB backend (`Ankit-Edureach3`). The existing test module relies on parsing static test and answer-key PDFs directly into MongoDB collections (`Test.js` and `TestAttempt.js`). While adequate for initial prototypes, this design cannot support dynamic paper generation from a verified question bank, complex JEE Advanced partial-marking matrices, millisecond-accurate timing telemetry, or high-performance enterprise analytics.

This masterplan designs a production-grade **Dual-Database Architecture**:

- **MongoDB (Existing Platform Backbone):** Continues to serve as the system of record for student authentication (`User.js` with `tokenVersion`), Razorpay purchases, mentorship enrolments (`Enrollment.js` generating `CP-YYYY-XXXXX`), mentor identity (`Mentor.js`), and daily study logs.
- **PostgreSQL 16+ (Dedicated Assessment Engine):** Introduces an ACID-compliant relational data store for academic taxonomies (Subjects, Chapters, Topics), question bank versioning, bilingual translations (Hindi/English with LaTeX equations), test blueprints, student exam attempts, question visit events, and evaluation metrics.
- **Redis & BullMQ (Real-Time Buffer & Worker Queue):** Manages server-authoritative timers, transient high-frequency autosave ingestion, and asynchronous background grading/email dispatch.
- **Microsoft Power BI (Executive & Academic BI):** Directly queries a dedicated PostgreSQL Read Replica modeled in an optimized Star Schema (Fact and Dimension views), delivering actionable insights without degrading transactional exam performance.

> **Guiding Architectural Principle**
>
> Zero disruption to existing users, authentication flows, or CP-ID sequences. MongoDB remains unaffected for core platform services, while the examination system operates with strict transactional integrity and analytical depth.

02 Recommended Technology Stack with Technical Rationale

A rigorous evaluation was conducted between PostgreSQL 16 and MySQL 8.0 for the dedicated examination database. PostgreSQL is recommended decisively for the following technical reasons:

CRITERIA	POSTGRESQL 16+ (RECOMMENDED)	MYSQL 8.0
Complex JEE Marking	**Superior:** Native `JSONB` with expression indexing allows storing variable section rules, partial marking maps, and bilingual option arrays with direct indexing.	Basic JSON support without comparable indexing power; requires rigid schemas or slower unindexed queries.
Cohort Analytics	**Built-in:** Advanced window functions (`DENSE_RANK()` , `PERCENT_RANK()` , `CUME_DIST()`) compute batch percentiles natively in SQL.	Window functions available but historically slower on large datasets; lacks specialized percentile aggregates.
High-Volume Partitioning	Declarative table partitioning by range or hash on `student_responses` and `timing_events` prevents B-Tree index bloat.	Partitioning is restrictive with foreign keys and requires rigid primary key compositions.
Locking & Concurrency	Supports `SELECT ... FOR UPDATE SKIP LOCKED` , allowing lock-free background queue consumption and zero race conditions during paper generation.	Supports skip-locked but exhibits more frequent metadata lock contention during high concurrency.

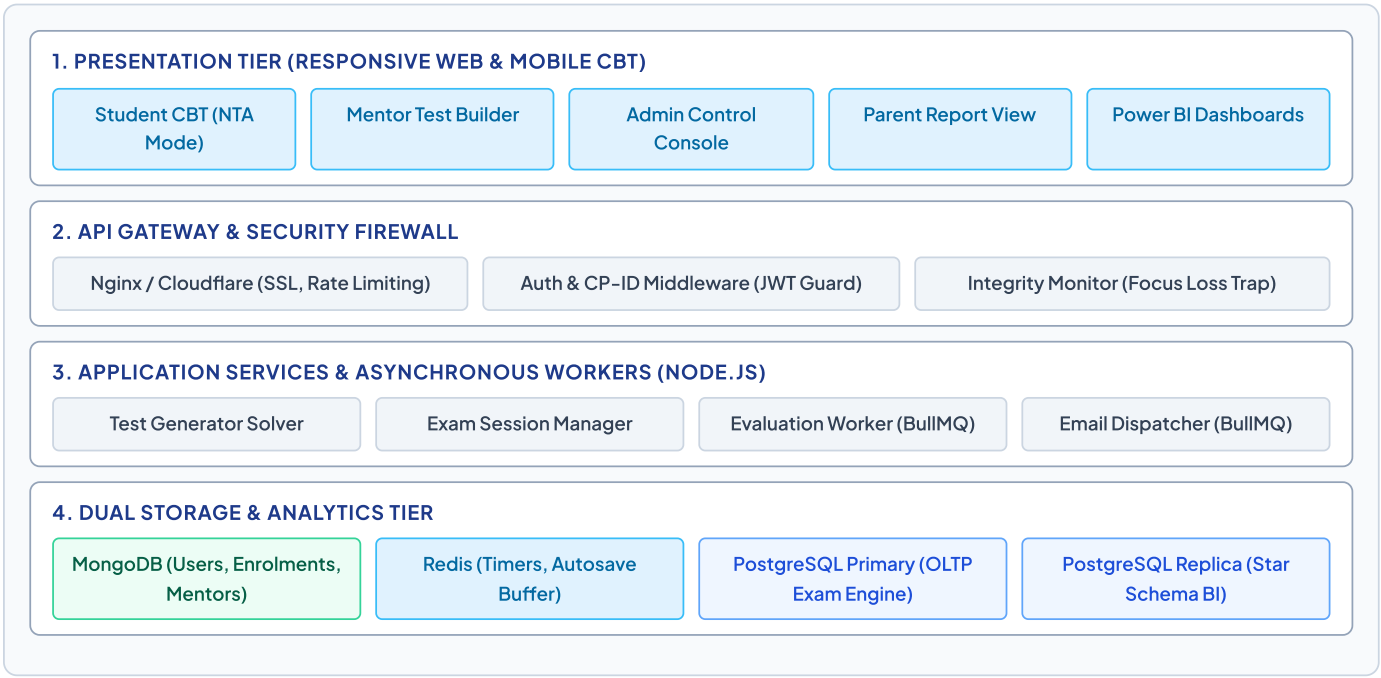
In-Memory Cache: Redis 7.0+

Enforces server-authoritative countdowns (independent of student device clocks), buffers rapid autosave beacons to prevent database write bottlenecks, and manages distributed locks to prevent double submissions.

Equation Rendering: KaTeX

Renders complex mathematical and scientific equations up to 100× faster than MathJax, with zero layout shift on mobile viewports and seamless offline rehydration.

03 System Architecture Overview



04 End-to-End Operational Workflow

The system automates the lifecycle of an examination across mentors, students, parents, and administrative leadership:

Step 1: Test Blueprint & Generation

Mentors configure exam pattern (JEE Main or Advanced), duration, chapter weightages, and difficulty ratios (e.g. 30% Easy, 50% Medium, 20% Hard). The stratified solver generates an immutable paper snapshot from the verified question bank.

Step 2: Publishing & Batch Assignment

The mentor reviews the paper blueprint. Upon approval, the paper is assigned to targeted batches (e.g. `mentor-jee-2027`). Scheduled start and cutoff times are published to student dashboards.

Step 3: Secure CBT Exam Attempt

Students log in with CP-ID. The server issues a cryptographic exam token and sets an authoritative countdown. Every answer switch and timestamp is buffered in Redis and mirrored in the browser's IndexedDB.

Step 4: Automated Grading & Diagnostics

Upon submission or timer expiry, BullMQ workers grade answers against the immutable paper key. The engine calculates positive marks, negative marks, net scores, chapter accuracy, and flags weak topics.

Step 5: Automated Multichannel Delivery & BI Ingestion

1. Results are saved in PostgreSQL.
2. Summary scores are mirrored to the student's MongoDB study log.
3. Transactional emails with secure links are dispatched to student and parent inboxes.
4. Replicated data feeds Power BI Star Schema views for mentor, batch, and organizational reporting.

05 MongoDB versus SQL Responsibility Matrix

DOMAIN ENTITY	MONGODB RESPONSIBILITY	POSTGRESQL RESPONSIBILITY	SYNC MECHANISM
Identity & Authentication	**Primary Owner:** `User.js` stores names, emails, phone numbers, password hashes, and `tokenVersion`.	Read-only external reference: `student_cp_id VARCHAR(32)`. Zero authentication logic in SQL.	JWT claims pass `cpId` into SQL queries.
Enrolments & Batches	**Primary Owner:** `Enrollment.js` assigns CP-IDs (`CP-2026-00042`), handles Razorpay payments, and manages plans.	Maintains `students_sync` table mirroring active student CP-IDs, batch IDs, and parent emails.	Post-save hook on MongoDB `Enrollment` executes upsert in SQL.
Question Bank	Deprecated legacy PDF parser. Unstructured questions migrated to SQL.	**Primary Owner:** Subjects, chapters, topics, bilingual LaTeX statements, options, assets, verification.	One-time migration script.
Test Configuration	None.	**Primary Owner:** Blueprints, difficulty quotas, section marking rules, immutable paper snapshots.	Native SQL.
Sessions & Timing	None.	**Primary Owner:** Attempt lifecycles, millisecond-level timing events, focus-loss logs.	Redis write-behind buffer.
Results & Scorecards	Cached summary scorecard in student study log for mentor review.	**Primary Owner:** Detailed question-level responses, partial marks, accuracy, weak-topic flags.	Async evaluation worker writes to SQL & updates Mongo.
Power BI Analytics	None. (Direct BI queries would degrade core web apps).	**Primary Owner:** Star Schema Views (`Fact*` , `Dim*`) hosted on PostgreSQL Read Replica.	On-Premises Data Gateway.

06 SQL Database Schema & ER Design

The schema enforces strict referential integrity, supports immutable versioning, and partitions high-frequency response tables.

```sql
-- Canonical DDL Highlights: schema.sql
CREATE TYPE exam_category_enum AS ENUM ('jee_main', 'jee_advanced', 'both');
CREATE TYPE question_type_enum AS ENUM ('single_mcq', 'multiple_mcq', 'numerical');

CREATE TABLE questions (
    question_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    exam_category exam_category_enum NOT NULL DEFAULT 'both',
    subject_id SMALLINT REFERENCES subjects(subject_id),
    chapter_id INT REFERENCES chapters(chapter_id),
    topic_id INT REFERENCES topics(topic_id),
    question_type question_type_enum NOT NULL,
    difficulty VARCHAR(16) NOT NULL, -- 'easy', 'medium', 'hard'
    numerical_correct_value NUMERIC(12, 4),
    numerical_tolerance NUMERIC(8, 4) DEFAULT 0.0000,
    partial_marking_rules JSONB DEFAULT NULL,
    verification_status VARCHAR(16) DEFAULT 'verified',
    current_version INT NOT NULL DEFAULT 1
);

CREATE TABLE question_translations (
    translation_id SERIAL PRIMARY KEY,
    question_id UUID REFERENCES questions(question_id) ON DELETE CASCADE,
    language_code VARCHAR(5) NOT NULL, -- 'en', 'hi'
    statement_latex TEXT NOT NULL,
    solution_latex TEXT,
    UNIQUE(question_id, language_code)
);

CREATE TABLE exam_attempts (
    attempt_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    paper_id UUID REFERENCES test_papers(paper_id),
    student_cp_id VARCHAR(32) NOT NULL REFERENCES students_sync(student_cp_id),
    status VARCHAR(24) DEFAULT 'in_progress',
    started_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    server_deadline TIMESTAMPTZ NOT NULL,
    focus_loss_count SMALLINT DEFAULT 0,
    UNIQUE(paper_id, student_cp_id)
);

CREATE TABLE student_responses (
    response_id BIGSERIAL,
    attempt_id UUID REFERENCES exam_attempts(attempt_id) ON DELETE CASCADE,
    paper_question_id INT NOT NULL,
    selected_option_keys CHAR(1)[],
    numerical_response NUMERIC(12, 4),
    status VARCHAR(24) DEFAULT 'unanswered',
    time_spent_seconds INT DEFAULT 0,
    PRIMARY KEY (response_id, attempt_id)
) PARTITION BY HASH (attempt_id);

CREATE TABLE student_responses_p0 PARTITION OF student_responses FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE student_responses_p1 PARTITION OF student_responses FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE student_responses_p2 PARTITION OF student_responses FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE student_responses_p3 PARTITION OF student_responses FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

07 Automated Paper Generation Algorithm

The paper generator implements a **Multi-Constraint Stratified Sampling Solver**. It guarantees exact question counts, enforces target difficulty ratios, and minimizes repeating recently exposed questions to the same student cohort.

```python
# Python Pseudocode: Stratified Paper Generator
def generate_balanced_paper(blueprint_id, target_batch_id):
    blueprint = db.get_blueprint(blueprint_id)
    rules = db.get_rules(blueprint_id)

    # 1. Target difficulty counts
    target_easy = round(blueprint.total_q * (blueprint.pct_easy / 100))
    target_med  = round(blueprint.total_q * (blueprint.pct_med / 100))
    target_hard = blueprint.total_q - target_easy - target_med
    quota = {"easy": target_easy, "medium": target_med, "hard": target_hard}

    # 2. Exclude questions seen by this batch in the past 60 days
    exposed_ids = db.get_recently_attempted_qids(target_batch_id, days=60)
    selected_questions = []

    for rule in rules:
        candidates = db.query_verified_questions(
            subject=rule.subject_id, chapter=rule.chapter_id,
            category=blueprint.category, types=rule.allowed_types
        )

        if len(candidates) < rule.required_count:
            raise InsufficientQuestionsException(f"Shortage in Chapter {rule.chapter_id}")

        # Ranking function: Prioritize unexposed + deficit difficulty matching
        def score(q):
            unexposed_bonus = 100 if q.id not in exposed_ids else 0
            difficulty_deficit = quota[q.difficulty] * 10
            return unexposed_bonus + difficulty_deficit + random.random()

        candidates.sort(key=score, reverse=True)
        chosen = candidates[:rule.required_count]

        for q in chosen:
            quota[q.difficulty] -= 1
            selected_questions.append(q)

    with db.transaction():
        paper_id = db.create_immutable_paper_snapshot(blueprint, selected_questions)
    return paper_id
```

08 NTA-Inspired Examination Interface

The CBT interface recreates the authentic testing ergonomics of official computer-based examinations, providing clear focus, rapid navigation, and accessible visual indicators.

NTA Standard 5-State Question Palette

PALETTE STATE	VISUAL INDICATOR	BEHAVIORAL DEFINITION
Not Visited	01 Silver Gray	Question has not been opened or viewed yet during this attempt.
Not Answered	02 Bold Red	Question was viewed, but student navigated away without choosing an option.
Answered	03 Forest Green	Question has a saved response; counted towards evaluation.
Marked for Review	04 Violet Circle	Student flagged question for later reconsideration; no answer selected.
Ans & Marked Review	05 Violet + Green	Question answered AND flagged for review. *Evaluated in final scoring.*

Zero-Data-Loss Autosave Guarantee

Every answer selection is written simultaneously to React local state, persisted in the browser's `IndexedDB`, and flushed via `fetch(..., { keepalive: true })` to Redis. If a student's browser closes unexpectedly or the network disconnects, reopening the page rehydrates their exact state within 500ms.

09 Exam Integrity & Anti-Cheating Architecture

Campus Point implements a realistic, best-effort browser integrity policy that acknowledges web platform limitations without making false claims of absolute lockdown.

1st Focus Loss

Mild Warning: The exam freezes, enters full-screen overlay, and alerts the student: *"Warning 1/2: Navigation away from the test window is logged."* Student must click to acknowledge.

2nd Focus Loss

Severe Warning: Amber alert: *"Final Warning 2/2: The next focus loss event will terminate your exam immediately."*

3rd Focus Loss

Auto-Submission: Exam locks immediately. Status set to `terminated_malpractice`. All previously saved responses are graded as-is.

Honest Technical Limitation Disclaimer

Standard web browsers cannot prevent a student from using an adjacent phone or secondary machine. If ironclad lockdown is required, a dedicated Safe Exam Browser (SEB) with OS-level virtualization controls must be introduced in a future phase.

10 Automatic Evaluation Formulas & Worked Examples

The evaluation engine enforces strict separation between positive marks, negative marks, and net scores across standard JEE Main and variable JEE Advanced schemes:

Core Evaluation Equations

$$ ext{Positive Marks} = sum ext{MarksEarned}, quad ext{Negative Marks} = sum | ext{Penalties}|, quad ext{Net Score} = ext{Positive} - ext{Negative}$$ $$ ext{Accuracy %} = left(rac{ ext{Correct Answers Count}}{ ext{Total Questions Attempted}} ight) imes 100, quad ext{Percentage Score} = left(rac{ ext{Net Score}}{ ext{Max Marks}} ight) imes 100$$

Numerical Verification Case Study: JEE Main Full Practice Paper

SUBJECT	TOTAL QS	ATTEMPTED	CORRECT	INCORRECT	UNATTEMPTED	POSITIVE	NEGATIVE	NET SCORE	ACCURACY
Physics	25	20	16	4	5	+64.00	4.00	**+60.00**	80.0%
Chemistry	25	22	19	3	3	+76.00	3.00	**+73.00**	86.4%
Mathematics	25	18	13	5	7	+52.00	5.00	**+47.00**	72.2%
Overall Total	**75**	**60**	**48**	**12**	**15**	**+192.00**	**12.00**	**+180.00**	**80.0%**

11 Student Report Design (Deep-Blue Theme)

Modeled after the reference screenshots in `src/components/mentor/StudentReport.jsx`, the report pairs an authoritative deep-blue banner with white diagnostic cards.

CAMPUS POINT JEE MAIN FULL TEST 04 — **CANDIDATE: CP-2026-00042**

180 / 300	80.0%	94.2 %tile	#08 / 142	60 / 75
NET SCORE	ACCURACY	BATCH PERCENTILE	BATCH RANK	QUESTIONS ATTEMPTED

Time Efficiency Quadrant

- **Fast & Correct (Mastered):** 38 Qs (Avg 1.2 min/Q)
- **Slow & Correct (Needs Speed):** 10 Qs (Avg 3.8 min/Q)
- **Fast & Wrong (Careless Guess):** 7 Qs (Avg 0.9 min/Q) ⚠
- **Slow & Wrong (Concept Gap):** 5 Qs (Avg 4.1 min/Q) ⚠

Priority Revision Areas (Diagnostic Flags)

- **Rotation Dynamics:** 3 Wrong | -15 Marks Lost | Acc: 25%
- **Definite Integration:** 2 Wrong | -10 Marks Lost | Acc: 33%
- **P-Block Elements:** 2 Wrong | -10 Marks Lost | Acc: 0%

12 Multi-Persona Dashboards

Mentor Dashboard

Provides one-click paper blueprint generation, real-time batch attempt tracking, unattempted student alerts, topic-lag heatmaps, and direct task assignment to `MentorTask.js`.

Student Dashboard

Displays active/scheduled exams, past performance trend lines, review room with bilingual KaTeX solutions, and personalized revision priority queues.

Parent Dashboard

Presents jargon-free weekly progress, punctuality scores, net score trends, and clear mentor action items accessible via tokenized magic links.

Admin Console

Monitors question bank coverage by chapter, mentor test creation velocity, outlier analysis on ambiguous questions, and overall platform uptime.

13 REST API Specification

METHOD	ENDPOINT	AUTHORIZED ROLES	DESCRIPTION & IDEMPOTENCY
`POST`	`/api/v2/exam/blueprints`	Mentor, Admin	Create blueprint with chapter weightages and difficulty quotas.
`POST`	`/api/v2/exam/blueprints/:id/generate`	Mentor, Admin	Execute stratified solver; return immutable draft paper.
`POST`	`/api/v2/exam/papers/:id/publish`	Mentor, Admin	Publish paper and bind to student batch plans.
`POST`	`/api/v2/exam/attempts/start`	Student	Initialize exam session, lock authoritative server deadline.
`GET`	`/api/v2/exam/attempts/:id/paper`	Student	Fetch questions without answer keys; bilingual LaTeX payloads.
`PUT`	`/api/v2/exam/attempts/:id/autosave`	Student	Idempotent batch update of answers and timing telemetry.
`POST`	`/api/v2/exam/attempts/:id/submit`	Student	Final submission lock; enqueues BullMQ evaluation worker.
`GET`	`/api/v2/exam/results/:attemptId`	Student, Mentor	Retrieve evaluated scorecard, topic breakdown, and solutions.
`GET`	`/api/v2/exam/reports/parent`	Public (Tokenized)	Secure, token-authorized parent performance scorecard.

14 Microsoft Power BI Architecture & Star Schema

Power BI connects to a dedicated PostgreSQL Read Replica via an On-Premises Data Gateway to prevent analytical aggregation queries from impacting active exam autosaves.

Star Schema Architecture

- **Fact Tables:** `bi_fact_test_attempts` (Grain: 1 row/student attempt), `bi_fact_question_responses` (Grain: 1 row/question response).
- **Dimension Tables:** `DimStudent`, `DimBatch`, `DimMentor`, `DimTest`, `DimDate`, `DimSubject`, `DimChapter`, `DimTopic`, `DimDifficulty`.

```
-- Essential Power BI DAX Measures
AverageNetScore = AVERAGE(bi_fact_test_attempts[net_score])

CohortAccuracyRate =
DIVIDE(
    SUM(bi_fact_test_attempts[correct_count]),
    SUM(bi_fact_test_attempts[attempted_questions]),
    0
) * 100

QuestionDiscriminationIndex =
VAR Top27Threshold = PERCENTILE.INC(bi_fact_test_attempts[net_score], 0.73)
VAR Bottom27Threshold = PERCENTILE.INC(bi_fact_test_attempts[net_score], 0.27)
VAR TopPassRate = CALCULATE(AVERAGE(bi_fact_question_responses[is_correct]), bi_fact_test_attempts[net_score] ≥ Top27Threshold)
VAR BottomPassRate = CALCULATE(AVERAGE(bi_fact_question_responses[is_correct]), bi_fact_test_attempts[net_score] ≤ Bottom27Thre
RETURN TopPassRate - BottomPassRate
```

15 Security, Reliability & Scalability Plan

- **Server-Authoritative Clock:** Client timers are visual mirrors. Deadlines are verified in SQL and cached in Redis. Submissions past deadline are rejected or automatically forced.
- **Double-Submission Prevention:** Submissions acquire a Redis distributed lock (`redlock:submit:{attemptId}`) with an atomic SQL status transition (`status = 'in_progress' → 'submitted'`).
- **Database Protection via Redis Streams:** Autosave requests are acknowledged instantly via Redis streams and flushed asynchronously to PostgreSQL in micro-batches (500 records/flush), supporting up to 10,000 concurrent students.
- **Disaster Recovery:** Point-in-time recovery (PITR) with continuous WAL archiving to Cloudflare R2 / AWS S3 every 15 minutes.

16 Phased Implementation Roadmap

PHASE	TARGET DELIVERABLES	PRIORITY	KEY RISK & MITIGATION
Phase 1	PostgreSQL 16 setup, DDL schema execution, MongoDB CP-ID sync bridge.	P0 REQUIRED	*Risk:* Data drift between databases. *Mitigation:* Nightly reconciliation cron.
Phase 2	Question bank schema, bilingual LaTeX editor, KaTeX pipeline.	P0 REQUIRED	*Risk:* LaTeX parsing errors on mobile. *Mitigation:* Strict KaTeX client validation.
Phase 3	Mentor Blueprint Builder, Stratified Paper Generation Solver.	P1 STANDARD	*Risk:* Question deficit in niche topics. *Mitigation:* Fallback constraint relaxation.
Phase 4	NTA-mode CBT interface, IndexedDB offline persistence, Focus-loss traps.	P0 REQUIRED	*Risk:* Network disconnection during exam. *Mitigation:* Local storage auto-rehydration.
Phase 5	Evaluation Engine (JEE Main + Advanced partial marks), Result persistence.	P0 REQUIRED	*Risk:* Edge-case partial marking bugs. *Mitigation:* Comprehensive unit test suite.
Phase 6	Deep-blue student scorecard, Parent magic link portal, BullMQ emails.	P1 STANDARD	*Risk:* Mail provider rate limits. *Mitigation:* Rate-limited BullMQ worker queue.
Phase 7	Power BI Star Schema Views, Data Gateway, Executive Dashboards.	P2 ANALYTICS	*Risk:* OLAP queries locking OLTP tables. *Mitigation:* Route BI to Read Replica only.
Phase 8	K6 load testing (5,000 concurrent attempts), Production cutover.	P0 REQUIRED	*Risk:* CPU spike during simultaneous submits. *Mitigation:* PgBouncer connection pooling.

17 Testing Strategy & Production-Readiness Checklist

Targeted Automated Test Suites

- **Paper Generation Accuracy:** Verifies exact difficulty distributions and topic weights across 500 test generations.
- **Partial Marking Verification:** Unit tests covering all multi-correct permutations (+4, +3, +2, +1, -2, 0).
- **Autosave Reconnection:** Simulates browser kill and tests IndexedDB rehydration under active countdowns.
- **Load Testing:** K6 simulation of 5,000 students submitting answers every 15 seconds ($p95 < 120 ext{ms}$).

Production Launch Go/No-Go Checklist

- [ ] PostgreSQL 16 connection pooling configured via PgBouncer.
- [ ] MongoDB Enrollment post-save hook successfully syncs CP-IDs.
- [ ] KaTeX equations render with zero layout shift on mobile devices.
- [ ] Server-authoritative countdown forcibly submits expired tests.
- [ ] Focus loss traps trigger warnings at count 1 and 2, auto-submit at 3.
- [ ] Power BI Gateway queries routed strictly to PostgreSQL read replica.

18 & 19 Project Structure & Initial Migration Steps

18. Recommended Project Structure

```
Ankit-Edureach3/
├── server/
│   ├── index.js              # Server bootstrap
│   ├── db.js                 # Existing MongoDB client
│   ├── sql/
│   │   ├── db.js             # PostgreSQL pg pool
│   │   └── schema.sql        # Canonical DDL
│   ├── routes/exam/          # Exam REST API
│   │   ├── blueprints.js
│   │   ├── attempts.js
│   │   └── results.js
│   ├── services/exam/
│   │   ├── generator.js      # Paper solver
│   │   └── evaluator.js      # Grading engine
│   └── workers/
│       └── evalWorker.js     # BullMQ consumer
└── src/
    └── components/exam/      # React NTA Interface
        ├── ExamHeader.jsx
        ├── QuestionPalette.jsx
        └── QuestionViewer.jsx
```

19. Initial Migration Sequence

1. **Database Setup:** Provision PostgreSQL 16 and apply `schema.sql`.
2. **Sync Hook Deployment:** In `server/models/Enrollment.js`, register a post-save hook to write active CP-IDs to `students_sync` in SQL.
3. **Historical Backfill:** Execute `scripts/backfill_students_to_sql.js` to populate existing student enrolments into PostgreSQL.
4. **Taxonomy Seeding:** Seed standard JEE Physics, Chemistry, and Mathematics chapters and topics into PostgreSQL.
5. **Connection Pool Tuning:** Configure `max: 50`, `idleTimeoutMillis: 30000` in `server/sql/db.js`.

20 Prioritized List of Immediate Tasks

TIMELINE	ACTION ITEMS	SUCCESS CRITERIA
Week 1	Deploy PostgreSQL 16, execute `schema.sql`, configure `server/sql/db.js`, implement MongoDB-to-SQL sync bridge for CP-IDs.	Enrolling a student in MongoDB instantly creates an associated row in PostgreSQL `students_sync`.
Week 2	Deploy bilingual question bank CRUD APIs, KaTeX math renderer, and implement `generator.js` paper generation solver.	Mentors can create a test blueprint and preview a generated 75-question balanced paper in <1.5 seconds.
Week 3	Build React NTA CBT exam interface with 5-state palette, IndexedDB local persistence, and server autosave routes.	Simulated tab closures and network dropouts restore active student answers with zero data loss.
Week 4	Deploy evaluation engine, BullMQ email dispatcher, Deep-Blue student scorecard, and configure Power BI Star Schema views.	Submitting an exam immediately evaluates scores, updates dashboards, sends parent emails, and updates Power BI views.

Separation of Concerns: Confirmation & Future Scope

Confirmed Core Requirements: Dual-database architecture (MongoDB + PostgreSQL), CP-ID preservation, JEE Main & Advanced support, bilingual Hindi/English, NTA 5-state palette, focus-loss tracking, deep-blue student reports, Power BI Star Schema.

Recommended Solutions: PostgreSQL 16+ over MySQL for native JSONB and window functions; Redis streams for autosave buffering; KaTeX for math rendering; Read Replica for Power BI gateway isolation.

Assumptions for Confirmation: PostgreSQL and Redis hosting platform (Managed RDS/Supabase vs Self-hosted VPS); Power BI deployment tier (Pro vs Premium); parent email capture rate.

Future Enhancements: Safe Exam Browser (SEB) desktop integration for proctored examinations; automated question OCR ingestion from PDF archives.